

1 Asymptotic Warm Up

Give the tightest asymptotic bound on `foo(n)`.

```
1 public int foo(int n) {  
2     if (n == 0) {  
3         return 0;  
4     }  
5     bloop(n);  
6     return foo(n / 3) + foo(n / 3) + foo(n / 3);  
7 }  
8  
9 public int bloop(int n) {  
10     for (int i = 0; i < n; i += 1) {  
11         System.out.println("Ah, loops too");  
12     }  
13     return n;  
14 }
```

$\Theta(N \log_3 N)$

2 Asymptotic Potpourri

Note: These are hard problems. If you are stuck on it for a long time, move on to other problems, and post on Ed or come to Office Hours so we can help you.

For the following methods, give the runtime of in Θ notation. Your answer should be a function of N that is as simple as possible with no unnecessary leading constants or lower order terms.

- (a) Give the runtime of `mystery1(n)` in Θ notation.

```

1 public void mystery1(int n) {
2     for (int i = n; i > 0; i = i / 2) {
3         for (int j = 0; j < 100000000; j += 2) {
4             System.out.println("Hello World");
5         }
6     }
7 }

```

$\Theta(N)$

- (b) Give the runtime of `mystery2(n)` in Θ notation.

```

1 public void mystery2(int n) {
2     for (int i = 1; i < n; i += 1) {
3         for (int j = 0; j < n; j += 1) {
4             i = i * 2;
5             j = j * 2;
6         }
7     }
8 }

```

$\Theta(\log N)$

- (c) What sum represents the work done by `mystery3(n)`? No need to simplify the sum, just write out the first few terms and the last term.

```

1 public void mystery3(int n) {
2     for (int i = n; i > 0; i = i / 2) {
3         for (int j = 1; j < i * i; j *= 2) {
4             System.out.println("Hello World");
5         }
6     }
7 }

```

$\Theta(N)$

Handwritten work for (c):

1024	4096	2^5
512	256	$2^4 \cdot \sqrt{2}$
256	16	2^4
	8	2^6
	4	2^4

- (d) Give the runtime of `mystery4(n)` in Θ notation. Assume that the `SLList` constructor, and the `size` and `addFirst` methods take constant time.

```

1 public void mystery4(int n) {
2     SLList<Integer> list = new SLList<>();
3     for (int i = 1; list.size() < n; i += 1) {
4         for (int j = 0; j < i; j += 1) {
5             list.addFirst(j);
6         }
7         System.out.print(list.size() + " + ");
8     }
9 }

```

$\Theta(N)$

3 WQU

- (a) Draw the Weighted Quick Union object on 0 through 10, that results from the following connect calls. Do not use path compression. What is the resulting underlying array? If we connect two sets of equal weight, we will tie-break by making the set whose root has a larger number the parent of the other (the opposite tie-breaking scheme as discussion 6).

connect(0, 1);

connect(2, 3);

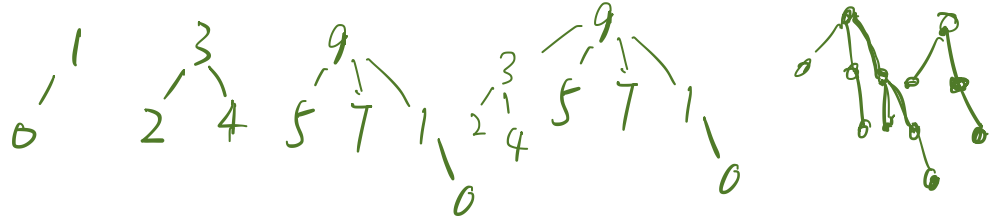
connect(9, 5);

connect(5, 7);

connect(7, 1);

connect(4, 2);

connect(3, 1);



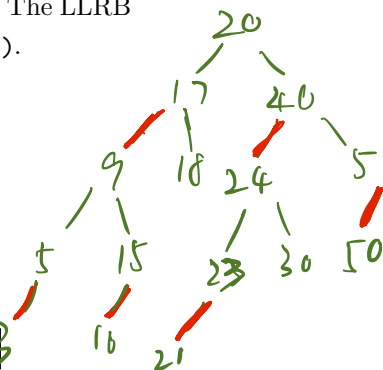
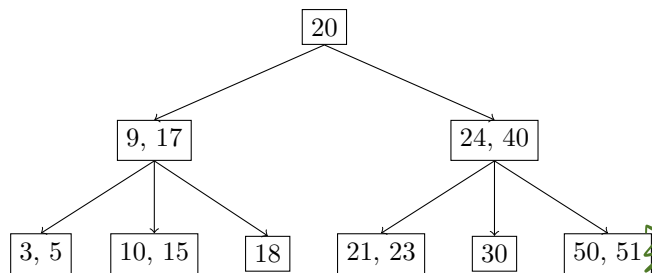
- (b) Assume that a single node has a height of 0. What are the shortest and tallest heights for a Quick Union object with 16 connected elements? What about for a Weighted Quick Union object? $\rightarrow$ 1 and 4

- (c) What are the best and worst runtimes for connect and isConnected in a Quick Union object with N connected elements? What about in a Weighted Quick Union object?

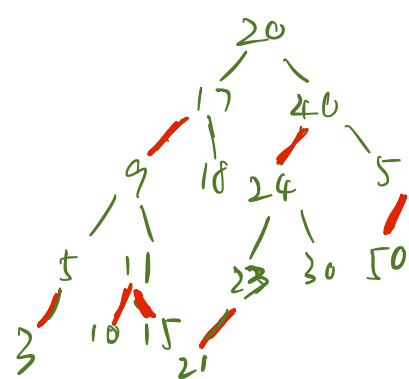
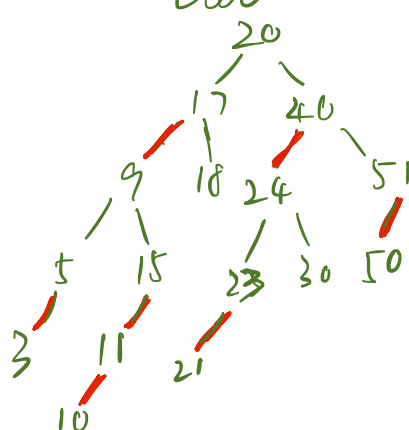
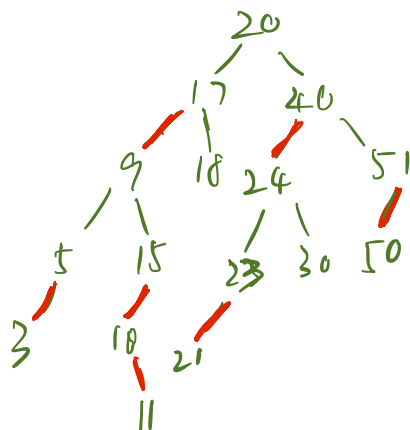
	best	worst
QU	$\Theta(1)$	$\Theta(N)$
WQU	$\Theta(1)$	$\Theta(\log N)$

4 Switcheroo

- (a) Consider the following 2-3 tree. Convert it to an LLRB, and describe the 6 LLRB operations to balance the tree after inserting the number 11. The LLRB operations are: `rotateRight(x)`, `rotateLeft(x)`, and `colorFlip(x)`.

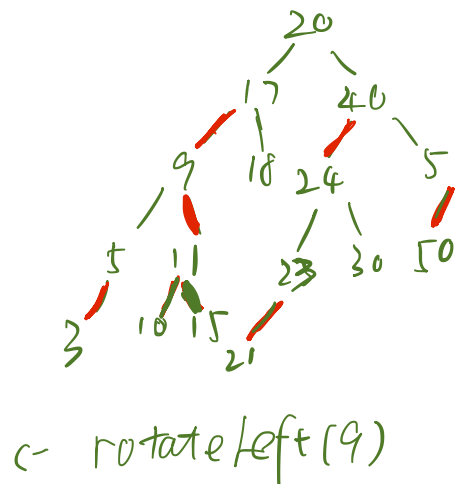
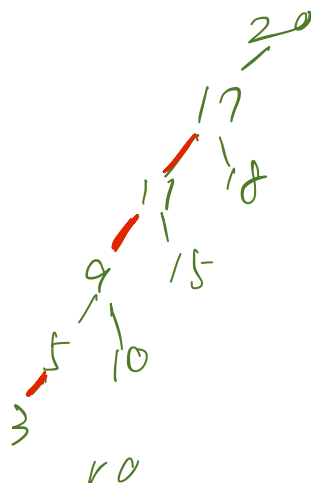
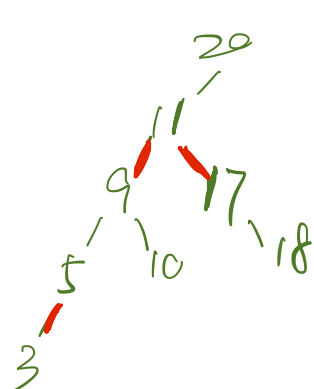


- (b) After inserting 11 and balancing the LLRB, how many nodes are on along the longest path from the root to a leaf. *from three to two*
- (c) After inserting 11 and balancing the LLRB, how many red links are on along the longest path from the root to a leaf. *two*

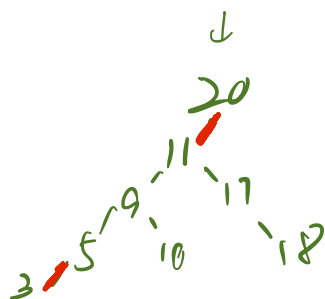


rotateLeft(10)

→ rotateRight(15) → flip(11)



c- rotateLeft(9)



5 Mechanical Hashing

Suppose we insert the following words into an initially empty hash table, in this order: **kerfuffle**, **broom**, **hroom**, **ragamuffin**, **donkey**, **brekky**, **blob**, **zenzizenzenic**, and **yap**. Assume that the hash code of a String is just its length (note that this is not actually the hash code for Strings in Java). Use separate chaining to resolve collisions. Assume 4 is the initial size of the hash table's internal array, and double this array's size when the load factor is equal to 1. Illustrate this hash table with a box-and-pointer diagram.

For each index of the final hash table, specify what Strings are stored in it. If it is empty, write "none".

